1. General Information

PostgreSQL is the primary relational database management system (DBMS) used in the AI Business Assistant project.

It provides persistent storage for user accounts, company information, business data and AI conversation history.

Technical Specifications

Parameter	**Value**
DBMS	PostgreSQL
Port	5432
Protocol	PostgreSQL Wire Protocol
Query Language	SQL
Data Format	Relational / JSONB
Encoding	UTF-8
Containerization	Docker

2. Database Architecture

The project uses a single PostgreSQL instance with logically separated schemas.

PostgreSQL

```
│
├── auth
│   └── users
│
└── core
├── companies
├── company_members
├── chats
├── messages
└── business_data
```

Each schema represents a separate logical domain and has its own responsible microservice.

- auth — managed by Auth Service.
- core — managed by Core Service.

3. Database Schema

3.1. Schema: auth

Table: users

Purpose: storage of user accounts and authentication credentials.

Column	Type	Constraints
id	SERIAL	PRIMARY KEY
name	VARCHAR(255)	NOT NULL
email	VARCHAR(255)	UNIQUE, NOT

		NULL
password_hash	VARCHAR(255)	NOT NULL
created_at	TIMESTAMP	DEFAULT NOW()

User registration and authentication are handled exclusively by Auth Service.

Passwords are stored as secure hashes using Argon2id or bcrypt.

3.2. Schema: core

Table: companies

Purpose: storage of information about companies managed by users.

Column	Type	Constraints
id	SERIAL	PRIMARY KEY
name	VARCHAR(255)	NOT NULL
description	TEXT	
industry	VARCHAR(255)	
created_at	TIMESTAMP	DEFAULT NOW()

Table: company_members

Purpose: association between users and companies, including access roles.

Column	Type	Constraints
id	SERIAL	PRIMARY KEY
user_id	INTEGER	NOT NULL
company_id	INTEGER	FK → companies(id)
role	VARCHAR(50)	NOT NULL

Possible roles:

- owner
- admin
- member

A UNIQUE constraint is applied to the combination of user_id and company_id.

user_id references the corresponding user in the auth.users schema.

Table: chats

Purpose: storage of conversations between users and AI Assistant.

Column	Type	Constraints
id	SERIAL	PRIMARY KEY
user_id	INTEGER	NOT NULL
company_id	INTEGER	FK → companies(id)

title	VARCHAR(255)	
created_at	TIMESTAMP	DEFAULT NOW()

Each conversation belongs to a user and is associated with a company context.

Table: messages

Purpose: storage of conversation history.

Column	Type	Constraints
id	BIGSERIAL	PRIMARY KEY
chat_id	INTEGER	FK → chats(id)
role	VARCHAR(20)	NOT NULL
content	TEXT	NOT NULL
created_at	TIMESTAMP	DEFAULT NOW()

Possible role values:

- user — user message.
- assistant — AI response.
- system — system instruction.

Messages are stored in chronological order.

Table: business_data

Purpose: storage of structured business information used

by AI Service for analysis and recommendations.

Column	Type	Constraints
id	SERIAL	PRIMARY KEY
company_id	INTEGER	FK → companies(id)
data_type	VARCHAR(100)	NOT NULL
data	JSONB	NOT NULL
created_at	TIMESTAMP	DEFAULT NOW()
updated_at	TIMESTAMP	DEFAULT NOW()

Possible data_type values:

- financial — financial indicators.
- sales — sales information.
- marketing — marketing indicators.
- employees — employee information.

JSONB allows storing business information with different structures without creating a separate table for every data type.

4. Entity Relationships

Relation	Type
Users → Company_Members	One-to-Many
Companies → Company_Members	One-to-Many

Users → Chats	One-to-Many
Companies → Chats	One-to-Many
Chats → Messages	One-to-Many
Companies → Business_Data	One-to-Many

5. Database Access and Microservice Responsibilities

PostgreSQL is accessed directly by Auth Service and Core Service.

Auth Service

Responsible for:

- User registration.
- User authentication.
- Retrieving user account information.
- Managing authentication-related data.

Accesses the auth schema.

Core Service

Responsible for:

- Company management.
- Company membership and roles.
- Chat management.
- Message history.

- Business data management.

Accesses the core schema.

AI Service

AI Service does not directly access PostgreSQL.

All database operations are performed through Core Service.

Communication flow:

1. Core Service receives a request from Frontend.
2. Core Service retrieves the necessary business context from PostgreSQL.
3. Core Service sends an AI processing task to RabbitMQ.
4. AI Service receives the task.
5. AI Service processes the request and generates a response.
6. The result is returned to Core Service through RabbitMQ.
7. Core Service saves the generated response in PostgreSQL.
8. Frontend receives the result.
9. Data Storage Principles
10. PostgreSQL is the primary persistent storage of the project.
11. User passwords are stored exclusively as secure

hashes.

12. Conversation history is stored in chats and messages.
13. Business information is stored using JSONB.
14. Database relationships are maintained using foreign keys where applicable.
15. Frequently queried fields should have appropriate indexes.
16. Auth Service and Core Service have separate logical database schemas.
17. AI Service accesses business information through Core Service rather than connecting directly to PostgreSQL.
18. Future Extensions

The database can be extended as the project evolves.

Extension	**Purpose**
pgvector	Vector similarity search for RAG
knowledge_chunks	Storage of text fragments and their embeddings
tasks	Storage of business tasks
ai_reports	Storage of AI-generated reports
integrations	External service integrations
subscriptions	User subscription management